

ULPF — Universal Log Pre-processing Framework

A high-throughput ingestion and normalization pipeline for security telemetry, with cryptographic integrity guarantees.

Brief for faculty review — Big Data Analytics | Prepared by the project team

1. The problem we were given

NTRO (with NCIIPC, which protects critical infrastructure — power, banking, telecom) asked for a framework that ingests logs from *any* perimeter network device — firewalls, intrusion detectors, proxies, web servers — and converts them into one common schema, at a scale of **billions of events per day**.

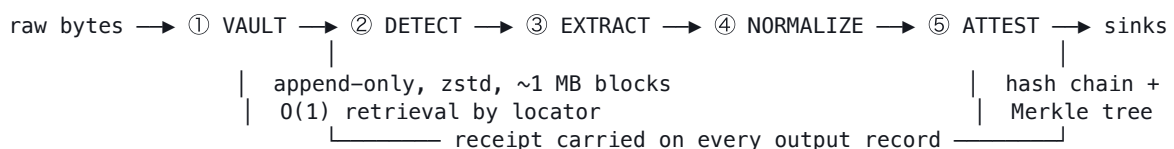
Two things make it harder than a normal ETL problem:

- **No schema on read, and no schema on write either.** Every vendor invents its own line format. There are ~40 formats in a typical estate and no registry of them.
- **The pipeline must be lossless in the forensic sense.** The statement asks to "preserve complete raw event data without information loss" and "maintain traceability between normalized and original events." These logs become legal evidence, so a transformation that drops unrecognised fields is a failure even if the analytics still work.

It must also run **air-gapped** — no internet, no cloud service, no external API.

2. What we built

A single Rust binary implementing a five-stage pipeline. The stage ordering is the main design decision:



The design decision worth your attention

The raw write precedes parsing. Bytes are appended to an immutable store and a locator returned *before* any interpretation is attempted. Everything downstream is therefore allowed to fail — unknown vendor, malformed record, a bug in our own decoder — without data loss. Most pipelines do the reverse: parse, then persist what was understood.

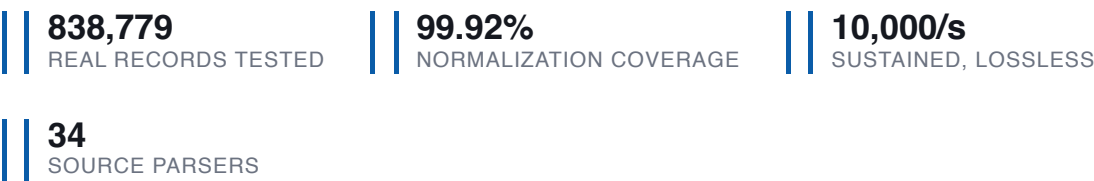
Schema and parser model

- **Target schema:** OCSF 1.9 — an open, vendor-neutral security event schema (the alternatives, ECS/CIM/ASIM/UDM, are each controlled by a single vendor).
- **Parsers are declarative data, not code.** One YAML file per source declares its recognition predicates, a decoder chain, and a field-to-schema mapping. Adding a source is a config change, hot-reloaded by a filesystem watcher — no recompile, no restart.
- **Ten composable decoders** (syslog ×2 dialects, key-value, CSV, CEF, LEEF, JSON, XML, regex) chained per source. Vendors don't invent formats; they wrap one of ~6 bodies in one of ~2 envelopes.

Outputs

- NDJSON (default), OpenSearch bulk, Splunk HEC
- **Hive-partitioned Parquet**, plus a separate **24-column feature table with a versioned contract** for model training — deliberately decoupled from the document archive so a parser change cannot silently alter the training schema

3. Where we are — measured



All corpora are public research datasets (HoneyNet Project, LogHub) used unmodified; the measurement script is in the repository and reproducible. No synthetic data contributes to any published figure.

Offered rate	Sent	Ingested	Loss
10,000 EPS		100,000	100,0000%
12,000 EPS		120,000	118,0001.7%
15,000 EPS		150,000	125,00016.7%

We publish 10,000 EPS as the sustained figure and refuse the 12,000 row, since a 1.7% drop is exactly the failure mode the design exists to prevent. **10,000 EPS = 864M events/day, which is 86% of the one-billion target.**

THE BOTTLENECK, STATED PRECISELY

The pipeline is **single-writer by construction**: one append-only store (so a locator is stable), one hash chain (so record $N+1$ links to N), one Merkle log (so an inclusion proof is meaningful). Parallelising within a collector means either multiple chains behind one process — which is sharding with extra steps — or a lock around the chain, which re-serialises the work.

Measured limits: **~10,000 EPS** on the UDP receive path, **~23,000 EPS** on the per-record processing path from file. So the receive path binds first.

Our answer is horizontal: N independent collectors, each with its own port, store, signing key and chain. We have verified two collectors ingesting concurrently with zero loss and each chain verifying independently against its own key. Five would reach 50,000 EPS.

4. Limitations we already know about

Limitation	Detail
Scaling is arithmetic, not measurement	The 5-shard → 50,000 EPS figure is the single-node number multiplied by shard count. Shards share no lock, chain or file, so linear scaling is the model's prediction — but we have never run a genuine multi-host test. <i>This is our weakest claim.</i>
Nine of 34 parsers lack real-corpus evidence	They pass tests written from vendor documentation. We have direct evidence this is insufficient: one vendor's published field order was simply <i>wrong</i> , and our manual-derived parser matched none of 8.1M real records until we rebuilt it from the log file's own header.
UDP has no backpressure	Above the sustained rate, records are dropped in the kernel before we observe them. We raise the socket buffer to 8 MB and report what was granted, but TCP/TLS ingest is unbuilt.
No binary formats	NetFlow/IPFIX and Windows EVTX are unsupported. Our decoder trait takes <code>&str</code> ; binary needs a second interface taking <code>&[u8]</code> . NetFlow is genuinely in scope, so this is a real gap.
Console has no authentication	Origin and Host checks block cross-site and DNS-rebinding attacks, but that is browser safety, not access control.
Feature table lacks a content digest	Rows record which parser produced them, but a parser edited without a version bump is indistinguishable from its predecessor — so a training set is reproducible only as far as the parser files are unchanged.

5. Where your input would help most

Four things, roughly in order of how much they would improve the submission. The first is by far the most valuable.

① A multi-machine throughput measurement

We need 3–5 networked machines for roughly two hours: one generating load, the others running collectors, measuring aggregate ingest and per-shard loss.

Why it matters: this converts our only unmeasured claim into a measured one. Is there a lab, a cluster, or a set of VMs we could use? We can bring the whole setup as a single binary and a compose file — no installation required on the machines.

② A review of our partitioning decision

We chose single-writer-per-shard and partition by *source device* (a device's traffic must not be split, or its records land in two chains). Ordering exists within a shard but not across shards.

Why we're asking you: this is the classic partitioning trade-off from your domain. Is device-level partitioning the right key, or would you argue for something else? Are we wrong that per-shard sequential integrity forces single-writer?

③ A critique of the feature table design

24 fixed columns, Hive-partitioned by date, every column required, absence encoded as a documented sentinel (-1 for numbers, empty string for text) rather than null — chosen because a null costs the reader a branch per row.

Why we're asking you: you train models on tabular data. Is a required-only schema with sentinels the right call, or would you prefer nullable columns? Are 24 columns the right features for anomaly detection on network telemetry, and what would you add?

④ Access to real device logs — a small, anonymised sample

Even **50–100 lines per device** is enough to write and verify a parser. We are *not* asking for production logs, and we understand why that would not be appropriate. Masked addresses and usernames are fine — we need the *format*, not the content.

*Even simpler and equally useful: just the **names of the network devices the campus runs**. That alone tells us which of our nine unproven parsers matter in practice and which we can drop.*

WHAT WE CAN SHOW IN TEN MINUTES

The system running live: heterogeneous logs from several real captures normalizing into one schema; an *invented* log format being ingested and its addresses and ports extracted with no parser at all; and a tamper test that alters one field on one stored record, after which verification reports **fingerprint mismatch: event content has been altered** — while the original bytes remain retrievable and provably different.

Source code, measurement scripts and full documentation: github.com/D3v4nshPat3l/ULPF · Every figure in this brief is reproducible with `python tools/measure_coverage.py`.